Desarrollo web en entorno cliente

Reto 1 · 2DAW · Inventario inteligente: organiza, controla y optimiza

Se evalúa la parte cliente (HTML + JavaScript + comunicación con el servidor) de la aplicación, según los RA del módulo recogidos del reto: **RA1, RA5, RA6 y RA7**.

Pesos y escala

RA	Descripción	Puntos
RA1	Arquitecturas y tecnologías de programación en clientes web	10
RA5	Aplicaciones web interactivas: manejo de eventos	25
RA6	Modelo de objetos del documento (DOM) **y HTML**	30
RA7	Comunicación asíncrona cliente-servidor	35
	Total	**100**

RA1 — Arquitecturas y tecnologías en el cliente (10 puntos)

Indicador	Pts	Excelente (100 %)	Adecuado (75 %)	Básico (50 %)	Insuficiente (25 %)
1.1 Organización del código cliente	5	Separa HTML, CSS y JS; el JS está dividido en módulos/funciones con una responsabilidad (acceso a API, renderizado, eventos, sesión). Nombres claros, sin código duplicado ni funciones gigantes.	Separación correcta con alguna función que mezcla responsabilidades o algo de duplicación.	Ficheros separados, pero la lógica de API, DOM y eventos está mezclada en pocas funciones largas.	Todo en un único fichero/bloque, con JS o estilos incrustados en el HTML y sin estructura.
1.2 Selección y justificación de tecnologías	5	Justifica la elección (JS nativo o librería/framework, `fetch`, Web Storage o cookies para la sesión, control de versiones) y explica sus ventajas y límites en un cliente web.	Justifica la mayoría de las decisiones con argumentos correctos pero poco profundos.	Usa las tecnologías, pero solo explica *qué* hace, no *por qué* las eligió.	No justifica las decisiones o no distingue qué se ejecuta en cliente y qué en servidor.

RA5 — Aplicaciones web interactivas: eventos (25 puntos)

Indicador	Pts	Excelente (100 %)	Adecuado (75 %)	Básico (50 %)	Insuficiente (25 %)
5.1 Inicio y cierre de sesión	4	El evento `submit` del login usa `preventDefault`, valida usuario/contraseña antes de enviar, muestra el error de credenciales incorrectas, guarda la sesión y redirige según el rol (ADMIN/USER). El cierre de sesión la limpia.	Login y logout funcionan; falta validación previa o el mensaje de error es poco claro.	Login funciona, pero no gestiona credenciales incorrectas o no cierra bien la sesión.	El formulario recarga la página, no distingue roles o el login no funciona.
5.2 Formularios CRUD con validación (usuarios, equipamientos, ubicaciones)	6	Los tres formularios gestionan crear, editar y eliminar con eventos. Validan campos obligatorios (nombre, apellidos, usuario, contraseña, rol; nombre, marca, modelo, categoría; nombre y descripción) y piden confirmación antes de eliminar.	Los tres CRUD funcionan; la validación o la confirmación es incompleta en alguno.	Solo uno o dos CRUD tienen eventos completos, o no hay validación en cliente.	Solo existe alguna operación aislada o no responde a los eventos.
5.3 Filtrado de equipamientos por categoría	4	El evento `change` del selector (cargado con las categorías del enum) filtra el inventario al instante, incluye la opción «todas» y funciona igual para ADMIN y USER.	El filtro funciona; falta la opción «todas» o hay algún defecto al combinarlo con otras acciones.	Filtra, pero requiere un botón/recarga o falla con alguna categoría.	El filtro no funciona o solo está disponible para un rol.
5.4 Asignación de ubicación (Conserje)	4	El Conserje cambia la ubicación de un equipamiento mediante evento (selector o botón), con confirmación del cambio, y el ADMIN no puede editarla (en la creación o en la edición).	La asignación funciona; falta confirmación o feedback, o la restricción del ADMIN es incompleta.	Cambia la ubicación, pero no impide la acción al rol incorrecto.	La asignación no funciona o cualquier rol puede hacerla.
5.5 Eventos de las notificaciones	5	El USER crea, edita y elimina sus notificaciones; el ADMIN solo cambia el estado. Los botones de editar y eliminar quedan deshabilitados o ausentes si la notificación está REALIZADA, y el USER no puede cambiar el estado.	Eventos y restricciones correctos, salvo un caso límite (por ejemplo, el estado REALIZADA en algún flujo).	Funcionan las acciones, pero las restricciones por rol o estado son incompletas.	Faltan acciones clave o no se respetan las restricciones.
5.6 Gestión técnica de eventos	2	Usa `addEventListener` y delegación de eventos en listas generadas dinámicamente; no duplica listeners tras re-renderizar; usa `event.target`/`dataset` de forma adecuada.	Buen uso general, con algún listener repetido o selector frágil.	Usa atributos `onclick` en línea o añade listeners nuevos en cada renderizado.	Los eventos de elementos dinámicos dejan de funcionar o provocan comportamiento errático.

RA6 — DOM y HTML (30 puntos)

HTML

Indicador	Pts	Excelente (100 %)	Adecuado (75 %)	Básico (50 %)	Insuficiente (25 %)
6.1 HTML semántico y válido	6	Estructura con `header`, `nav`, `main`, `section`, `footer`; tablas con `thead`/`tbody`/`th`, listas y botones donde corresponde; jerarquía de encabezados correcta; sin errores en el validador W3C; `lang`, `charset`, `viewport` y `title` presentes.	Estructura semántica casi completa; errores o avisos menores del validador.	Uso mixto de etiquetas semánticas y `div`/`span` genéricos; varios errores de validación.	Casi todo con `div`; anidamientos incorrectos, etiquetas sin cerrar o atributos inválidos.
6.2 Formularios HTML	4	Cada campo tiene `label` asociada, `name`/`id`, tipo adecuado (`text`, `password`, `select`, `textarea`), atributos `required`/`minlength`/`pattern` cuando aplican, y agrupa con `fieldset` cuando procede.	Formularios correctos; falta alguna label, algún atributo o una agrupación.	Campos sin `label` o con tipos inadecuados (todo `text`); validación HTML ausente.	Entradas sin `form`, sin `name` o contraseña visible en texto plano.
6.3 Estructura base y reutilización de vistas	2	Las vistas (login, usuarios, equipamientos, ubicaciones, notificaciones) tienen una estructura base coherente (cabecera, menú, contenedor) y los contenedores del DOM tienen `id`/`class` descriptivos y estables.	Estructura coherente con pequeñas inconsistencias entre vistas.	Estructura distinta en cada vista o identificadores poco claros.	Marcado desordenado que dificulta el acceso a los elementos desde JS.

DOM

Indicador	Pts	Excelente (100 %)	Adecuado (75 %)	Básico (50 %)	Insuficiente (25 %)
6.4 Renderizado del inventario	5	Genera dinámicamente cada equipamiento con nombre, marca, modelo, categoría y ubicación; muestra mensaje si no hay resultados; usa `createElement`/plantillas (`template`) o plantillas literales de forma ordenada.	Muestra todos los campos; el estado vacío o la estructura generada es mejorable.	Muestra solo parte de los campos o el HTML está incrustado de forma confusa.	Los datos están fijos en el HTML o la lista no se genera correctamente.
6.5 Interfaz según rol	4	Muestra u oculta menús, botones y secciones según el rol (el USER no ve gestión de usuarios, ni crear/editar/eliminar equipamientos ni ubicaciones; el ADMIN no ve crear notificaciones). Se aplica al cargar cada vista, sin parpadeos.	Adaptación correcta, con algún elemento que sigue visible o aparece momentáneamente.	Solo adapta el menú; los botones de acción siguen visibles para ambos roles.	La interfaz es idéntica para ambos roles.
6.6 Actualización del DOM tras operaciones	4	Tras crear, editar, eliminar, filtrar o reasignar, la vista se actualiza sin recargar la página, sin duplicados ni elementos obsoletos, y conserva el filtro activo.	Se actualiza correctamente, con algún caso que exige repetir la acción.	Solo actualiza tras recargar o duplica elementos en algunas operaciones.	El DOM no refleja los cambios.
6.7 Vistas de ubicaciones, historial y notificaciones	4	Muestra la lista de ubicaciones, el **historial de ubicaciones** de cada equipamiento (ubicación y orden/fecha) y las notificaciones con título, descripción, fecha, estado (con distinción visual PENDIENTE / EN_PROCESO / REALIZADA) y usuario creador.	Las tres vistas funcionan; falta algún dato (por ejemplo, el creador o la fecha) o la distinción visual.	Faltan el historial o varios campos de las notificaciones.	Alguna de las vistas no se genera.

Indicador	Pts	Excelente (100 %)	Adecuado (75 %)	Básico (50 %)	Insuficiente (25 %)
6.8 Manipulación segura y eficiente del DOM	3	Inserta datos del servidor con `textContent`/`createElement` (o los escapa) para evitar inyección de HTML; minimiza reflujos (fragmentos, un solo volcado); no usa `document.write`.	Seguro en general; algún uso de `innerHTML` con datos sin escapar.	Usa `innerHTML` con datos de usuario de forma habitual; reconstruye la página entera para cada cambio.	Inserta datos sin ningún control o con `document.write`.

RA7 — Comunicación asíncrona cliente-servidor (35 puntos)

Indicador	Pts	Excelente (100 %)	Adecuado (75 %)	Básico (50 %)	Insuficiente (25 %)
7.1 Capa de acceso a la API	6	Concentra las peticiones en funciones/módulo reutilizables con `fetch` y `async/await`; usa el método HTTP adecuado (GET, POST, PUT/PATCH, DELETE), `Content-Type` JSON, `JSON.stringify` y `response.json()`; la URL base es configurable.	Funciones reutilizables correctas; algún método o cabecera mal elegido, o URLs repetidas.	`fetch` repartido por el código con duplicación, o mezcla de promesas encadenadas y callbacks sin criterio.	Peticiones mal formadas o uso de métodos incorrectos que impiden la comunicación.
7.2 Autenticación en las peticiones	6	Envía las credenciales/token/sesión en cada petición protegida; gestiona la caducidad o los 401/403 redirigiendo al login; no guarda la contraseña en el cliente; explica el riesgo de guardar datos en `localStorage` y la necesidad de validar los permisos también en el servidor.	Autenticación funcional; la gestión de 401/403 o la caducidad es incompleta.	Autentica el login, pero no incluye las credenciales en las peticiones posteriores o las gestiona de forma frágil.	La autenticación no funciona o guarda datos sensibles sin protección.
7.3 CRUD asíncrono de las tres entidades	7	Usuarios, equipamientos y ubicaciones se crean, consultan, modifican y eliminan contra el servidor; los datos enviados cumplen el modelo (identificador, campos obligatorios, rol ADMIN/USER, categoría del enum, etc.) y tras cada operación se usa la respuesta del servidor.	Las tres entidades funcionan con algún defecto en una operación.	Solo una o dos entidades tienen CRUD completo.	Solo hay consultas, o las modificaciones no llegan al servidor.
7.4 Consultas, filtrado y historial	4	Carga inicial del inventario y las ubicaciones desde el servidor; filtro por categoría (en el servidor mediante parámetros o en el cliente sobre los datos recibidos, justificando la decisión); consulta del historial de ubicaciones de un equipamiento.	Consultas y filtro correctos; el historial es incompleto o los parámetros están mal construidos.	Cargas y filtros parciales, o con datos de ejemplo mezclados con los reales.	No consulta los datos al servidor.
7.5 Gestión de errores	5	Controla `response.ok` y los códigos HTTP (400, 401, 403, 404, 409, 500) y los fallos de red con `try/catch`; muestra mensajes comprensibles al usuario; la aplicación sigue operativa tras un error.	Gestiona los errores principales; mensajes genéricos o algún caso sin controlar.	Solo captura errores de red, o los muestra únicamente en la consola.	Sin gestión: un error provoca fallos silenciosos o bloquea la interfaz.
7.6 Estados de carga y sincronización	4	Indica la carga (spinner o botón deshabilitado), evita envíos duplicados, no deja condiciones de carrera, usa `Promise.all` cuando carga datos en paralelo y mantiene la interfaz coherente con el servidor.	Gestiona carga y duplicados en los casos principales.	Solo muestra un indicador de carga en algunas vistas; permite dobles envíos.	Sin control: la interfaz puede mostrar datos obsoletos o contradictorios.

Indicador	Pts	Excelente (100 %)	Adecuado (75 %)	Básico (50 %)	Insuficiente (25 %)
7.7 Reglas de negocio reflejadas en las peticiones	3	Respeta en el cliente las reglas del reto: al crear un equipamiento no se envía otra ubicación que no sea Almacén; las notificaciones nuevas se crean como PENDIENTE y asociadas al usuario autenticado; no se permite eliminar ni modificar las REALIZADA; el USER no puede cambiar el estado.	Cumple la mayoría; falla en un caso límite.	Cumple parte de las reglas; otras solo se ocultan en la interfaz sin controlarlas.	No aplica ninguna de las reglas del reto.